

Prueba 4 - Variantes de ensamblado de modelos

Este notebook hace lo siguiente:

1. Carga los modelos entrenados del pipeline SOB4ES: 3 familias single-target (Ridge, RF, XGBoost) y 5 familias multisalida (RF, XGBoost, RegressorChain, MLP, MLP con loss personalizada).
2. Ejecuta el smoke test habitual sobre **todo eval.csv** (no una muestra).
3. Separa `eval.csv` en meta-train/holdout, y calcula sobre el meta-train los pesos, ranking y coeficientes necesarios para las variantes que se calibran con datos.
4. Compara, todas sobre el mismo holdout, **5 formas de combinar los modelos base**:
 - **Media simple**: peso igual para todos.
 - **Media ponderada (R2)**: pesos proporcionales al R2 de cada modelo.
 - **Top-k**: media simple, pero solo entre los `K_TOP` mejores modelos.
 - **Mediana**: mas robusta a un modelo atípico (como Ridge) que la media.
 - **Stacking convexo**: pesos $w \geq 0$, $\sum(w) = 1$, optimizados en meta-train.

Por que hay un holdout (y no se evalua todo sobre `eval.csv` completo)

La media simple y la mediana no calibran nada con datos, asi que podrian evaluarse sobre las 65 filas de `eval.csv` sin problema. Pero la media ponderada, el top-k y el stacking convexo **si calibran algo con datos** (pesos, ranking, coeficientes): si se calibran y evaluaran sobre las mismas filas, el resultado quedaria inflado (el metodo "ha visto" la respuesta correcta de antemano). Para que las 5 variantes sean comparables entre si en igualdad de condiciones, todas se evaluan sobre el mismo holdout que no se ha usado para calibrar nada.

1. Configuración y carga de librerías

- `MODELS_DIR`: carpeta donde estan guardados los modelos entrenados.
- `EVAL_PATH`: ruta a `eval.csv`.
- `FEATURES_AUTORIZADAS`: las 34 variables de entrada (mismo orden que en el resto del pipeline).
- `TARGETS_ORDER`: los 21 targets **en el mismo orden** en que los modelos multisalida fueron entrenados (necesario para poder indexar su salida). Si no lo recuerdas de memoria, miralo en el notebook de entrenamiento multisalida (columna `y` usada en `.fit`).
- `TARGETS_TO_TEST`: el subconjunto de targets con los que se prueba este ensamblado (por defecto los dos prioritarios + uno mas a elegir).
- `SHOW_ENSEMBLE_MEMBERS`: si es `True`, la seccion 3b imprime el R2 de cada miembro individual de cada familia multisalida antes de promediar (util para depurar). No afecta a los resultados finales, es solo informativo.

```
In [1]: import json
import warnings
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.optimize import minimize
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("Librerias cargadas correctamente...")

MODELS_DIR = Path("output/models")
EVAL_PATH = Path("input/eval.csv")

TARGETS_ORDER = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

FEATURES_AUTORIZADAS = [
    'total_plant_cover_z',
    'clay_content_z',
```

```

'silt_content_z',
'sand_content_z',
'aggregate_stability_z',
'bulk_density_z',
'soil_moisture_z',
'as_z',
'cu_z',
'k_z',
'mo_z',
'ni_z',
'p_z',
'pb_z',
'zn_z',
'soil_ph_z',
'plot_total_organic_c_z',
'plot_total_n_z',
'gee_temp_media_C_z',
'gee_humedad_rel_pct_z',
'gee_ndvi_verano_z',
'dem_elevacion_m_z',
'dem_pendiente_deg_z',
'dem_orientacion_deg_z',
'eu_clay_content_z',
'eu_sand_content_z',
'eu_silt_content_z',
'eu_water_holding_capacity_z',
'eu_cn_ratio_z',
'eu_p_z',
'eu_ph_z',
'eu_as_z',
'eu_organic_carbon_octop_z',
'eu_zn_z'
]

# Targets con los que se prueba el ensamblado por media + otros métodos.
TARGETS_TO_TEST = [
    # Shannon
    'nematode_shannon_z',
    'macro_shannon_z',
    'earthworm_shannon_z',
    'orib_shannon_z',
    'meso_shannon_z',
    'coll_shannon_z',
    'bac_shannon_z',
    'fun_shannon_z',
    'euk_shannon_z',
    'oomy_shannon_z',
    'cerc_shannon_z',
    # Richness
    'macro_order_richness_z',
    'earthworm_richness_z',
    'orib_species_richness_z',
    'meso_species_richness_z',
    'coll_species_richness_z',
    'bac_asv_richness_z',
    'fun_asv_richness_z',
    'euk_asv_richness_z',
    'oomy_asv_richness_z',
    'cerc_asv_richness_z'
]

SHOW_ENSEMBLE_MEMBERS = False # True para depurar el detalle de cada miembro de cada familia

META_TEST_SIZE = 0.3 # proporcion de eval.csv reservada como holdout para evaluar las variantes
K_TOP = 4 # numero de modelos que se quedan en la variante top-k

assert all(t in TARGETS_ORDER for t in TARGETS_TO_TEST), (
    "Algun target de TARGETS_TO_TEST no esta en TARGETS_ORDER - revisa la configuracion."
)

```

Librerías cargadas correctamente...

2.- Carga de eval.csv

```

In [2]: eval_df = pd.read_csv(EVAL_PATH)

missing_features = [f for f in FEATURES_AUTORIZADAS if f not in eval_df.columns]
missing_targets = [t for t in TARGETS_ORDER if t not in eval_df.columns]
if missing_features:
    print(f"[AVISO] Faltan {len(missing_features)} features en eval.csv: {missing_features[:5]}...")
if missing_targets:
    print(f"[AVISO] Faltan {len(missing_targets)} targets en eval.csv: {missing_targets[:5]}...")

X_eval = eval_df[FEATURES_AUTORIZADAS]
y_eval = eval_df[TARGETS_ORDER]

print(f"eval.csv: {eval_df.shape[0]} filas, {eval_df.shape[1]} columnas")
print(f"X_eval: {X_eval.shape} | y_eval: {y_eval.shape}")

```

eval.csv: 65 filas, 71 columnas
X_eval: (65, 34) | y_eval: (65, 21)

3.- Definición y carga de los modelos base

Todos los modelos son ensamblados internos de varios miembros entrenados por separado, **salvo Ridge, que es un unico modelo por target (sin ensamblado)**. El resto - single-target (RF single-target, XGBoost single-target, un ensamblado por target) y multisalida (RF, XGBoost, RegressorChain, MLP, MLP custom loss, un ensamblado por familia) - si son ensamblados: 5 miembros en todos salvo RegressorChain, que tiene 10.

`EnsembleWrapper` carga todos los miembros disponibles de una familia (y, si es single-target, de un target concreto) y promedia su prediccion, de forma que el resto del notebook trata cada familia como un unico modelo ya consolidado:

- Si es **single-target**, cada miembro ya predice directamente el target → se promedian las predicciones tal cual.
- Si es **multisalida**, cada miembro predice los 21 targets a la vez → primero se selecciona la columna del target pedido en cada miembro y despues se promedia entre miembros.

Notas sobre el formato de cada tipo de modelo:

- **Ridge, RF y XGBoost** (single y multi) se guardan como `.pkl` y se cargan igual, con `joblib.load`.
- **MLP y MLP custom loss**: cada miembro tiene sus propios pesos (`.pt`, un `state_dict`), pero el config (`.pkl`) con la arquitectura usada al entrenar es **unico por familia** (no uno por miembro): todos los miembros de "MLP multisalida" comparten el mismo config, y lo mismo para "MLP custom loss".

Si un fichero (o un miembro del ensamblado) no existe o falla la carga, se **omite con un aviso** en vez de interrumpir el notebook, asi puedes ejecutar esto aunque todavia no tengas todos los modelos entrenados.

```
In [3]: def load_sklearn(path):
        return joblib.load(path)

def load_mlp_member(weights_path, config_path):
    import torch
    import torch.nn as nn

    config = joblib.load(config_path)

    def _cfg_get(key, default):
        # El config puede venir como dict o como un objeto (p.ej. dataclass/Namespace).
        if isinstance(config, dict):
            return config.get(key, default)
        return getattr(config, key, default)

    class GenericMLP(nn.Module):
        def __init__(self, input_dim, hidden_dims, output_dim, dropout=0.0):
            super().__init__()
            layers = []
            prev_dim = input_dim
            for h in hidden_dims:
                layers.append(nn.Linear(prev_dim, h))
                layers.append(nn.ReLU())
                if dropout > 0:
                    layers.append(nn.Dropout(dropout))
                prev_dim = h
            layers.append(nn.Linear(prev_dim, output_dim))
            self.red = nn.Sequential(*layers)

        def forward(self, x):
            return self.red(x)

        def predict(self, X):
            self.eval()
            with torch.no_grad():
                X_t = torch.as_tensor(np.asarray(X), dtype=torch.float32)
                return self.red(X_t).numpy()

    model = GenericMLP(
        input_dim=_cfg_get("input_dim", len(FEATURES_AUTORIZADAS)),
        hidden_dims=_cfg_get("hidden", [64, 32]),
        output_dim=_cfg_get("output_dim", len(TARGETS_ORDER)),
        dropout=_cfg_get("dropout", 0.0),
    )
    state_dict = torch.load(Path(weights_path), map_location="cpu")
    model.load_state_dict(state_dict)
    model.eval()
    return model

class EnsembleWrapper:
    '''Modelo (single-target o multisalida) guardado como ensamblado de varios miembros
    entrenados por separado. predict() promedia la prediccion de todos los miembros
    disponibles. Si falta algun miembro (fichero no encontrado, error de carga),
    simplemente se promedia con los que si se cargaron.

    kind="single": cada miembro predice directamente el target → se promedian tal cual.
    kind="multi": cada miembro predice todos los targets a la vez → primero se
    selecciona la columna del target pedido en cada miembro y despues se promedia.
    '''

    def __init__(self, key, label, members, kind, targets_order=None):
        self.key = key
        self.label = label
        self.members = members
        self.kind = kind
        self.targets_order = targets_order

    def predict(self, X, target=None):
        member_preds = []
        for model in self.members:
            preds = np.asarray(model.predict(X))
            if self.kind == "multi":
                if preds.ndim == 1:
                    preds = preds.reshape(-1, 1)
                idx = self.targets_order.index(target)
                preds = preds[:, idx]
            else:
```

```

preds = preds.reshape(-1)
member_preds.append(preds)
return np.mean(np.column_stack(member_preds), axis=1)

def load_ensemble_members(loader, path_template, n_members, loader_kwargs=None, **template_kwargs):
    '''Carga los n_members ficheros de un ensamblado, tolerando fallos parciales.
    loader_kwargs son argumentos fijos que se pasan a loader() en cada miembro (p.ej.
    un config compartido por toda la familia, en vez de uno por miembro).
    Devuelve (members, errors), donde errors es una lista de (índice, tipo_error, mensaje).
    ...
    loader_kwargs = loader_kwargs or {}
    members = []
    errors = []
    for i in range(n_members):
        path = Path(path_template.format(i=i, **template_kwargs))
        try:
            members.append(loader(path, **loader_kwargs))
        except Exception as e:
            errors.append((i, type(e).__name__, str(e)))
    return members, errors

SINGLE_MODEL_CONFIGS = [
    {"key": "ridge", "label": "Ridge", "loader": load_sklearn, "path_template": str(MODELS_DIR / "reg/ridge_prod_{}"), "n_members": 1},
    {"key": "rf_single", "label": "RF single-target", "loader": load_sklearn, "path_template": str(MODELS_DIR / "rf/rf_prod_{}"), "n_members": 1},
    {"key": "xgb_single", "label": "XGBoost single-target", "loader": load_sklearn, "path_template": str(MODELS_DIR / "xgb/xgb_prod_{}"), "n_members": 1}
]

MULTI_MODEL_CONFIGS = [
    {"key": "rf_multi", "label": "RF multisalida", "loader": load_sklearn, "path_template": str(MODELS_DIR / "rf_multi/rf_{}"), "n_members": 5},
    {"key": "xgb_multi", "label": "XGBoost multisalida", "loader": load_sklearn, "path_template": str(MODELS_DIR / "xgb_multi/xgb_{}"), "n_members": 5},
    {"key": "regchain", "label": "RegressorChain", "loader": load_sklearn, "path_template": str(MODELS_DIR / "chain/chain_{}"), "n_members": 1},
    {"key": "mlp_multi", "label": "MLP multisalida", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp/mlp_prod_{}"), "n_members": 5},
    {"key": "mlp_custom", "label": "MLP custom loss", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp_custom/mlp_custom_{}"), "n_members": 5},
    {"key": "mlp_custom_loss", "label": "MLP custom loss", "loader": load_mlp_member, "path_template": str(MODELS_DIR / "mlp_custom/mlp_custom_loss_{}"), "n_members": 5}
]

loaded_single = {} # (key, target) → EnsembleWrapper
loaded_multi = {} # key → EnsembleWrapper

for cfg in SINGLE_MODEL_CONFIGS:
    for target in TARGETS_TO_TEST:
        members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
                                                loader_kwargs=cfg.get("loader_kwargs"), target=target)
        if members:
            loaded_single[(cfg["key"], target)] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="single")
            print(f"OK {cfg['label']:22s} {target} cargado ({len(members)}/{cfg['n_members']} miembros)")
            if errors:
                print(f" [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][2]}")
        else:
            print(f"FALLO {cfg['label']:22s} {target} omitido → no se cargo ningun miembro (0/{cfg['n_members']})")
            if errors:
                print(f" → primer error: {errors[0][1]}: {errors[0][2]}")

for cfg in MULTI_MODEL_CONFIGS:
    members, errors = load_ensemble_members(cfg["loader"], cfg["path_template"], cfg["n_members"],
                                                loader_kwargs=cfg.get("loader_kwargs"))
    if members:
        loaded_multi[cfg["key"]] = EnsembleWrapper(cfg["key"], cfg["label"], members, kind="multi", targets_order=TARGETS_ORDER)
        print(f"OK {cfg['label']:22s} cargado ({len(members)}/{cfg['n_members']} miembros)")
        if errors:
            print(f" [AVISO] {len(errors)} miembro(s) no disponible(s), se promedia solo con el resto → primer error: {errors[0][2]}")
    else:
        print(f"FALLO {cfg['label']:22s} omitido → no se cargo ningun miembro (0/{cfg['n_members']})")
        if errors:
            print(f" → primer error: {errors[0][1]}: {errors[0][2]}")

print(f"\nModelos disponibles: {len(loaded_single)} single-target (por target) + {len(loaded_multi)} familias multisalida")

```

```

OK RF single-target [meso_shannon_z] cargado (5/5 miembros)
OK RF single-target [coll_shannon_z] cargado (5/5 miembros)
OK RF single-target [bac_shannon_z] cargado (5/5 miembros)
OK RF single-target [fun_shannon_z] cargado (5/5 miembros)
OK RF single-target [euk_shannon_z] cargado (5/5 miembros)
OK RF single-target [oomy_shannon_z] cargado (5/5 miembros)
OK RF single-target [cerc_shannon_z] cargado (5/5 miembros)
OK RF single-target [macro_order_richness_z] cargado (5/5 miembros)
OK RF single-target [earthworm_richness_z] cargado (5/5 miembros)
OK RF single-target [orib_species_richness_z] cargado (5/5 miembros)
OK RF single-target [meso_species_richness_z] cargado (5/5 miembros)
OK RF single-target [coll_species_richness_z] cargado (5/5 miembros)
OK RF single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK RF single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [nematode_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_shannon_z] cargado (5/5 miembros)
OK XGBoost single-target [macro_order_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [earthworm_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [orib_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [meso_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [coll_species_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [bac_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [fun_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [euk_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [oomy_asv_richness_z] cargado (5/5 miembros)
OK XGBoost single-target [cerc_asv_richness_z] cargado (5/5 miembros)
OK RF multitivalida cargado (5/5 miembros)
OK XGBoost multitivalida cargado (5/5 miembros)
OK RegressorChain cargado (10/10 miembros)
OK MLP multitivalida cargado (5/5 miembros)
OK MLP custom loss cargado (5/5 miembros)

```

Modelos disponibles: 63 single-target (por target) + 5 familias multitivalida

3b.- (Opcional) Inspeccion de miembros individuales de cada ensamblado

Solo se ejecuta si `SHOW_ENSEMBLE_MEMBERS = True` en la seccion 1. Util para depurar si un miembro concreto (de un modelo single-target o de una familia multitivalida) esta dando resultados muy distintos al resto antes de que se pierda esa informacion al promediar. No afecta a los resultados de las secciones siguientes.

```

In [4]: if SHOW_ENSEMBLE_MEMBERS:
        print("--- Modelos single-target ---")
        for (key, target), wrapper in loaded_single.items():
            y_true_t = y_eval[target].values
            print(f"\n{wrapper.label} [{target}] ({len(wrapper.members)} miembros cargados)")
            for i, model in enumerate(wrapper.members):
                preds = np.asarray(model.predict(X_eval)).reshape(-1)
                r2 = r2_score(y_true_t, preds)
                print(f"    miembro {i} | R2={r2:.4f}")

        print("\n--- Familias multitivalida ---")
        for key, wrapper in loaded_multi.items():
            print(f"\n{wrapper.label} ({len(wrapper.members)} miembros cargados)")
            for target in TARGETS_TO_TEST:
                idx = TARGETS_ORDER.index(target)
                y_true_t = y_eval[target].values
                for i, model in enumerate(wrapper.members):
                    preds = np.asarray(model.predict(X_eval))
                    if preds.ndim == 1:
                        preds = preds.reshape(-1, 1)
                    preds_t = preds[:, idx]
                    r2 = r2_score(y_true_t, preds_t)
                    print(f"    miembro {i} | {target:22s} | R2={r2:.4f}")
            else:
                print("SHOW_ENSEMBLE_MEMBERS = False → seccion omitida (cambia el flag en la seccion 1 para activarla).")

```

SHOW_ENSEMBLE_MEMBERS = False → seccion omitida (cambia el flag en la seccion 1 para activarla).

4.- Smoke test sobre eval.csv completo

Mismas comprobaciones que el smoke test habitual de los notebooks de modelo (predice sin error, sin NaN/Inf, tamaño de salida correcto), pero aquí sobre **todas** las filas de `eval.csv` en vez de una muestra pequeña. Se reutilizan las predicciones para las secciones siguientes, así no se predice dos veces. Cada familia multitivalida ya llega aquí como una única predicción promediada entre sus miembros.

```

In [5]: def smoke_test(wrapper, X, y_true, target):
        try:
            preds = wrapper.predict(X, target=target)
        except Exception as e:
            print(f"[SMOKE TEST] {wrapper.label:22s} | {target:22s} | ERROR al predecir → {type(e).__name__}: {e}")
            return None

        preds = np.asarray(preds, dtype=float).reshape(-1)
        shape_ok = preds.shape[0] == len(X)
        n_nan = int(np.isnan(preds).sum())

```

```

n_inf = int(np.isinf(preds).sum())
ok = shape_ok and n_nan == 0 and n_inf == 0
r2 = r2_score(y_true, preds) if ok else float("nan")
status = "OK" if ok else "REVISAR"
print(f"[SMOKE TEST] {wrapper.label:22s} | {target:22s} | filas={len(preds):5d} | "
      f"NaN={n_nan} | Inf={n_inf} | R2={r2: .4f} | {status}")
return preds if ok else None

# predictions[target][label] = np.array de predicciones (solo si paso el smoke test)
predictions = {t: {} for t in TARGETS_TO_TEST}

for target in TARGETS_TO_TEST:
    y_true_t = y_eval[target].values
    print(f"\n--- {target} ---")

    for (key, t), wrapper in loaded_single.items():
        if t != target:
            continue
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
        if preds is not None:
            predictions[target][wrapper.label] = preds

    for key, wrapper in loaded_multi.items():
        preds = smoke_test(wrapper, X_eval, y_true_t, target)
        if preds is not None:
            predictions[target][wrapper.label] = preds

```

--- nematode_shannon_z ---

[SMOKE TEST] Ridge	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0112	OK
[SMOKE TEST] RF single-target	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1366	OK
[SMOKE TEST] XGBoost single-target	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1637	OK
[SMOKE TEST] RF multisalida	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1545	OK
[SMOKE TEST] XGBoost multisalida	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2228	OK
[SMOKE TEST] RegressorChain	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1467	OK
[SMOKE TEST] MLP multisalida	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0829	OK
[SMOKE TEST] MLP custom loss	nematode_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0668	OK

--- macro_shannon_z ---

[SMOKE TEST] Ridge	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1257	OK
[SMOKE TEST] RF single-target	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2797	OK
[SMOKE TEST] XGBoost single-target	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2601	OK
[SMOKE TEST] RF multisalida	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2000	OK
[SMOKE TEST] XGBoost multisalida	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.3771	OK
[SMOKE TEST] RegressorChain	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2804	OK
[SMOKE TEST] MLP multisalida	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2396	OK
[SMOKE TEST] MLP custom loss	macro_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2704	OK

--- earthworm_shannon_z ---

[SMOKE TEST] Ridge	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2395	OK
[SMOKE TEST] RF single-target	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.5059	OK
[SMOKE TEST] XGBoost single-target	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.4946	OK
[SMOKE TEST] RF multisalida	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.4160	OK
[SMOKE TEST] XGBoost multisalida	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.5375	OK
[SMOKE TEST] RegressorChain	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.4876	OK
[SMOKE TEST] MLP multisalida	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.3538	OK
[SMOKE TEST] MLP custom loss	earthworm_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.3770	OK

--- orib_shannon_z ---

[SMOKE TEST] Ridge	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1137	OK
[SMOKE TEST] RF single-target	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1644	OK
[SMOKE TEST] XGBoost single-target	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1147	OK
[SMOKE TEST] RF multisalida	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1693	OK
[SMOKE TEST] XGBoost multisalida	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2275	OK
[SMOKE TEST] RegressorChain	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2069	OK
[SMOKE TEST] MLP multisalida	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2250	OK
[SMOKE TEST] MLP custom loss	orib_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.2552	OK

--- meso_shannon_z ---

[SMOKE TEST] Ridge	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.2499	OK
[SMOKE TEST] RF single-target	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.1895	OK
[SMOKE TEST] XGBoost single-target	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.1868	OK
[SMOKE TEST] RF multisalida	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.1178	OK
[SMOKE TEST] XGBoost multisalida	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.2861	OK
[SMOKE TEST] RegressorChain	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.1640	OK
[SMOKE TEST] MLP multisalida	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.4925	OK
[SMOKE TEST] MLP custom loss	meso_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.4289	OK

--- coll_shannon_z ---

[SMOKE TEST] Ridge	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.4998	OK
[SMOKE TEST] RF single-target	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.4168	OK
[SMOKE TEST] XGBoost single-target	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.3312	OK
[SMOKE TEST] RF multisalida	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.1953	OK
[SMOKE TEST] XGBoost multisalida	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.3229	OK
[SMOKE TEST] RegressorChain	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.6087	OK
[SMOKE TEST] MLP multisalida	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	1.2378	OK
[SMOKE TEST] MLP custom loss	coll_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	1.2015	OK

--- bac_shannon_z ---

[SMOKE TEST] Ridge	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.0812	OK
[SMOKE TEST] RF single-target	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0037	OK
[SMOKE TEST] XGBoost single-target	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0092	OK
[SMOKE TEST] RF multisalida	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0290	OK
[SMOKE TEST] XGBoost multisalida	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=-	0.0151	OK

[SMOKE TEST]	RegressorChain	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0288	OK
[SMOKE TEST]	MLP multitalida	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0334	OK
[SMOKE TEST]	MLP custom loss	bac_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.1218	OK

--- fun_shannon_z ---

[SMOKE TEST]	Ridge	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0161	OK
[SMOKE TEST]	RF single-target	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0337	OK
[SMOKE TEST]	XGBoost single-target	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0206	OK
[SMOKE TEST]	RF multitalida	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0127	OK
[SMOKE TEST]	XGBoost multitalida	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0502	OK
[SMOKE TEST]	RegressorChain	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0219	OK
[SMOKE TEST]	MLP multitalida	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.1246	OK
[SMOKE TEST]	MLP custom loss	fun_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0672	OK

--- euk_shannon_z ---

[SMOKE TEST]	Ridge	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0407	OK
[SMOKE TEST]	RF single-target	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1575	OK
[SMOKE TEST]	XGBoost single-target	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0912	OK
[SMOKE TEST]	RF multitalida	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1378	OK
[SMOKE TEST]	XGBoost multitalida	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0886	OK
[SMOKE TEST]	RegressorChain	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1624	OK
[SMOKE TEST]	MLP multitalida	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1451	OK
[SMOKE TEST]	MLP custom loss	euk_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1025	OK

--- oomy_shannon_z ---

[SMOKE TEST]	Ridge	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0857	OK
[SMOKE TEST]	RF single-target	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0998	OK
[SMOKE TEST]	XGBoost single-target	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0965	OK
[SMOKE TEST]	RF multitalida	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1252	OK
[SMOKE TEST]	XGBoost multitalida	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0620	OK
[SMOKE TEST]	RegressorChain	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0913	OK
[SMOKE TEST]	MLP multitalida	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.1362	OK
[SMOKE TEST]	MLP custom loss	oomy_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0429	OK

--- cerc_shannon_z ---

[SMOKE TEST]	Ridge	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	0.0155	OK
[SMOKE TEST]	RF single-target	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0404	OK
[SMOKE TEST]	XGBoost single-target	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0414	OK
[SMOKE TEST]	RF multitalida	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0041	OK
[SMOKE TEST]	XGBoost multitalida	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0330	OK
[SMOKE TEST]	RegressorChain	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0006	OK
[SMOKE TEST]	MLP multitalida	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0855	OK
[SMOKE TEST]	MLP custom loss	cerc_shannon_z	filas=	65	NaN=0	Inf=0	R2=	-0.0613	OK

--- macro_order_richness_z ---

[SMOKE TEST]	Ridge	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.1521	OK
[SMOKE TEST]	RF single-target	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.3058	OK
[SMOKE TEST]	XGBoost single-target	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.3082	OK
[SMOKE TEST]	RF multitalida	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2532	OK
[SMOKE TEST]	XGBoost multitalida	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.4013	OK
[SMOKE TEST]	RegressorChain	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.4069	OK
[SMOKE TEST]	MLP multitalida	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.3447	OK
[SMOKE TEST]	MLP custom loss	macro_order_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.3581	OK

--- earthworm_richness_z ---

[SMOKE TEST]	Ridge	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2789	OK
[SMOKE TEST]	RF single-target	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.5492	OK
[SMOKE TEST]	XGBoost single-target	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.5171	OK
[SMOKE TEST]	RF multitalida	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.4509	OK
[SMOKE TEST]	XGBoost multitalida	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.5893	OK
[SMOKE TEST]	RegressorChain	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.5359	OK
[SMOKE TEST]	MLP multitalida	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.4628	OK
[SMOKE TEST]	MLP custom loss	earthworm_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.4854	OK

--- orib_species_richness_z ---

[SMOKE TEST]	Ridge	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.0921	OK
[SMOKE TEST]	RF single-target	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2407	OK
[SMOKE TEST]	XGBoost single-target	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.1259	OK
[SMOKE TEST]	RF multitalida	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2386	OK
[SMOKE TEST]	XGBoost multitalida	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2800	OK
[SMOKE TEST]	RegressorChain	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2026	OK
[SMOKE TEST]	MLP multitalida	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2783	OK
[SMOKE TEST]	MLP custom loss	orib_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.2981	OK

--- meso_species_richness_z ---

[SMOKE TEST]	Ridge	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0959	OK
[SMOKE TEST]	RF single-target	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0453	OK
[SMOKE TEST]	XGBoost single-target	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0592	OK
[SMOKE TEST]	RF multitalida	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0255	OK
[SMOKE TEST]	XGBoost multitalida	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.1146	OK
[SMOKE TEST]	RegressorChain	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0400	OK
[SMOKE TEST]	MLP multitalida	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.2681	OK
[SMOKE TEST]	MLP custom loss	meso_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.2198	OK

--- coll_species_richness_z ---

[SMOKE TEST]	Ridge	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.7341	OK
[SMOKE TEST]	RF single-target	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.5461	OK
[SMOKE TEST]	XGBoost single-target	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.5803	OK
[SMOKE TEST]	RF multitalida	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.2920	OK
[SMOKE TEST]	XGBoost multitalida	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.6465	OK
[SMOKE TEST]	RegressorChain	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.5799	OK
[SMOKE TEST]	MLP multitalida	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-2.0859	OK
[SMOKE TEST]	MLP custom loss	coll_species_richness_z	filas=	65	NaN=0	Inf=0	R2=	-1.9873	OK

--- bac_asv_richness_z ---

[SMOKE TEST]	Ridge	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=	-0.0761	OK
[SMOKE TEST]	RF single-target	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.0004	OK
[SMOKE TEST]	XGBoost single-target	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.0014	OK
[SMOKE TEST]	RF multitalida	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.0044	OK
[SMOKE TEST]	XGBoost multitalida	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=	0.0155	OK

[SMOKE TEST]	RegressorChain	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0052	OK
[SMOKE TEST]	MLP multitalida	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0799	OK
[SMOKE TEST]	MLP custom loss	bac_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.1811	OK
--- fun_asv_richness_z ---								
[SMOKE TEST]	Ridge	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0087	OK
[SMOKE TEST]	RF single-target	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0144	OK
[SMOKE TEST]	XGBoost single-target	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0242	OK
[SMOKE TEST]	RF multitalida	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0068	OK
[SMOKE TEST]	XGBoost multitalida	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0662	OK
[SMOKE TEST]	RegressorChain	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0186	OK
[SMOKE TEST]	MLP multitalida	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0105	OK
[SMOKE TEST]	MLP custom loss	fun_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0069	OK
--- euk_asv_richness_z ---								
[SMOKE TEST]	Ridge	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0400	OK
[SMOKE TEST]	RF single-target	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.2360	OK
[SMOKE TEST]	XGBoost single-target	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1671	OK
[SMOKE TEST]	RF multitalida	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1903	OK
[SMOKE TEST]	XGBoost multitalida	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.2233	OK
[SMOKE TEST]	RegressorChain	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.2690	OK
[SMOKE TEST]	MLP multitalida	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.2470	OK
[SMOKE TEST]	MLP custom loss	euk_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1970	OK
--- oomy_asv_richness_z ---								
[SMOKE TEST]	Ridge	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1035	OK
[SMOKE TEST]	RF single-target	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1895	OK
[SMOKE TEST]	XGBoost single-target	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1776	OK
[SMOKE TEST]	RF multitalida	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1917	OK
[SMOKE TEST]	XGBoost multitalida	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1650	OK
[SMOKE TEST]	RegressorChain	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.2019	OK
[SMOKE TEST]	MLP multitalida	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0640	OK
[SMOKE TEST]	MLP custom loss	oomy_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0188	OK
--- cerc_asv_richness_z ---								
[SMOKE TEST]	Ridge	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1019	OK
[SMOKE TEST]	RF single-target	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1207	OK
[SMOKE TEST]	XGBoost single-target	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1227	OK
[SMOKE TEST]	RF multitalida	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1171	OK
[SMOKE TEST]	XGBoost multitalida	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0983	OK
[SMOKE TEST]	RegressorChain	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.1171	OK
[SMOKE TEST]	MLP multitalida	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2= 0.0285	OK
[SMOKE TEST]	MLP custom loss	cerc_asv_richness_z	filas=	65	NaN=0	Inf=0	R2=-0.0845	OK

5.- Particion meta-train / holdout

Las variantes de la seccion 7 (media ponderada, top-k, stacking convexo) **calibran algo con datos** (pesos, ranking, coeficientes), asi que evaluarlas sobre las mismas filas que se usan para calibrarlas inflaria el resultado (el metodo "ha visto" la respuesta correcta de antemano). Por eso se separan las filas de `eval.csv` en dos bloques: uno para calibrar (meta-train) y otro para evaluar todo -- modelos individuales y las 5 variantes de combinacion por igual -- (holdout). La mediana y la media simple no calibran nada, pero se evaluan tambien sobre el holdout para que la comparacion final sea homogenea.

```
In [6]: idx_all = np.arange(len(eval_df))
idx_train, idx_holdout = train_test_split(idx_all, test_size=META_TEST_SIZE, random_state=RANDOM_STATE)

print(f"Filas totales en eval.csv: {len(idx_all)}")
print(f"Filas para calibrar pesos/seleccion (meta-train): {len(idx_train)}")
print(f"Filas para evaluar todo (holdout): {len(idx_holdout)}")
```

```
Filas totales en eval.csv: 65
Filas para calibrar pesos/seleccion (meta-train): 45
Filas para evaluar todo (holdout): 20
```

6.- Calculo de pesos, ranking y coeficientes de stacking (sobre meta-train)

Para cada target:

- **R2 por modelo** en meta-train (base para las variantes ponderada y top-k).
- **Pesos de la media ponderada:** `max(R2, 0)` normalizado para sumar 1 (un modelo con R2 negativo en meta-train no debería restar, así que se recorta a 0 antes de normalizar).
- **Ranking para top-k:** los `K_TOP` modelos con mejor R2 en meta-train.
- **Coeficientes del stacking con pesos no negativos que suman 1:** se resuelve un problema de optimización (SLSQP) que minimiza el error cuadrático en meta-train sujeto a $w \geq 0$ y $\sum(w) = 1$.

```
In [7]: def fit_convex_weights(X_meta_train, y_meta_train):
'''Encuentra pesos  $w \geq 0$ ,  $\sum(w) = 1$ , que minimizan el MSE en meta-train.'''
n_models = X_meta_train.shape[1]
w0 = np.full(n_models, 1.0 / n_models)

def objective(w):
    preds = X_meta_train @ w
    return np.mean((preds - y_meta_train) ** 2)

constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1.0}]
bounds = [(0.0, 1.0)] * n_models

result = minimize(objective, w0, method="SLSQP", bounds=bounds, constraints=constraints)
if not result.success:
    print(f" [AVISO] Optimizacion de pesos no convergio ({result.message}); se usan pesos uniformes.")
    return w0
return result.x

variant_info = {} # target -> dict con model_labels, weights, top_k_labels, convex_weights
```



```

for target in TARGETS_T0_TEST:
    model_labels = list(predictions[target].keys())
    if not model_labels:
        print(f"[AVISO] {target}: no hay modelos disponibles, no se pueden calcular pesos.")
        continue

    X_full = np.column_stack([predictions[target][lbl] for lbl in model_labels])
    y_target = y_eval[target].values
    X_train, y_train = X_full[idx_train], y_target[idx_train]

    r2_train = {lbl: r2_score(y_train, X_train[:, i]) for i, lbl in enumerate(model_labels)}

    raw_weights = np.array([max(r2_train[lbl], 0.0) for lbl in model_labels])
    if raw_weights.sum() == 0:
        print(f"[AVISO] {target}: todos los modelos tienen  $R^2 \leq 0$  en meta-train, se usan pesos uniformes.")
        weighted_weights = np.full(len(model_labels), 1.0 / len(model_labels))
    else:
        weighted_weights = raw_weights / raw_weights.sum()

    k = min(K_TOP, len(model_labels))
    top_k_labels = sorted(model_labels, key=lambda lbl: r2_train[lbl], reverse=True)[:k]

    convex_weights = fit_convex_weights(X_train, y_train)

    variant_info[target] = {
        "model_labels": model_labels,
        "r2_train": r2_train,
        "weighted_weights": weighted_weights,
        "top_k_labels": top_k_labels,
        "convex_weights": convex_weights,
    }

    print(f"\n--- {target} ---")
    print("R2 en meta-train por modelo:", {k: round(v, 3) for k, v in r2_train.items()})
    print("Pesos media ponderada:      ", {lbl: round(w, 3) for lbl, w in zip(model_labels, weighted_weights)})
    print(f"Top-{k}:                        {top_k_labels}")
    print("Pesos stacking convexo:        ", {lbl: round(w, 3) for lbl, w in zip(model_labels, convex_weights)})

```

```

--- nematode_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.038, 'RF single-target': 0.146, 'XGBoost single-target': 0.173, 'RF multisalida': 0.178, 'XGB
oost multisalida': 0.201, 'RegressorChain': 0.144, 'MLP multisalida': 0.108, 'MLP custom loss': 0.067}
Pesos media ponderada: {'Ridge': np.float64(0.036), 'RF single-target': np.float64(0.138), 'XGBoost single-target': np.float64
(0.164), 'RF multisalida': np.float64(0.169), 'XGBoost multisalida': np.float64(0.19), 'RegressorChain': np.float64(0.137), 'MLP multi
salida': np.float64(0.103), 'MLP custom loss': np.float64(0.063)}
Top-4: ['XGBoost multisalida', 'RF multisalida', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.319), 'XGBoost multisalida': np.float64(0.681), 'RegressorChain': np.float64(0.0), 'MLP multisalida': n
p.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- macro_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.084, 'RF single-target': 0.181, 'XGBoost single-target': 0.146, 'RF multisalida': 0.107, 'XGB
oost multisalida': 0.305, 'RegressorChain': 0.151, 'MLP multisalida': 0.152, 'MLP custom loss': 0.158}
Pesos media ponderada: {'Ridge': np.float64(0.065), 'RF single-target': np.float64(0.141), 'XGBoost single-target': np.float64
(0.114), 'RF multisalida': np.float64(0.083), 'XGBoost multisalida': np.float64(0.237), 'RegressorChain': np.float64(0.118), 'MLP mult
isalida': np.float64(0.118), 'MLP custom loss': np.float64(0.123)}
Top-4: ['XGBoost multisalida', 'RF single-target', 'MLP custom loss', 'MLP multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(1.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.fl
oat64(0.0), 'MLP custom loss': np.float64(0.0)}

--- earthworm_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.306, 'RF single-target': 0.529, 'XGBoost single-target': 0.526, 'RF multisalida': 0.419, 'XGB
oost multisalida': 0.578, 'RegressorChain': 0.506, 'MLP multisalida': 0.427, 'MLP custom loss': 0.435}
Pesos media ponderada: {'Ridge': np.float64(0.082), 'RF single-target': np.float64(0.142), 'XGBoost single-target': np.float64
(0.141), 'RF multisalida': np.float64(0.112), 'XGBoost multisalida': np.float64(0.155), 'RegressorChain': np.float64(0.136), 'MLP mult
isalida': np.float64(0.115), 'MLP custom loss': np.float64(0.117)}
Top-4: ['XGBoost multisalida', 'RF single-target', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(1.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.fl
oat64(0.0), 'MLP custom loss': np.float64(0.0)}

--- orib_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.163, 'RF single-target': 0.23, 'XGBoost single-target': 0.171, 'RF multisalida': 0.234, 'XGB
oost multisalida': 0.331, 'RegressorChain': 0.293, 'MLP multisalida': 0.328, 'MLP custom loss': 0.35}
Pesos media ponderada: {'Ridge': np.float64(0.078), 'RF single-target': np.float64(0.11), 'XGBoost single-target': np.float64(0.
081), 'RF multisalida': np.float64(0.111), 'XGBoost multisalida': np.float64(0.158), 'RegressorChain': np.float64(0.14), 'MLP multisal
ida': np.float64(0.156), 'MLP custom loss': np.float64(0.166)}
Top-4: ['MLP custom loss', 'XGBoost multisalida', 'MLP multisalida', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.413), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.
float64(0.0), 'MLP custom loss': np.float64(0.587)}
[AVIS0] meso_shannon_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- meso_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.419, 'RF single-target': -0.449, 'XGBoost single-target': -0.44, 'RF multisalida': -0.295,
'XGBoost multisalida': -0.538, 'RegressorChain': -0.455, 'MLP multisalida': -0.449, 'MLP custom loss': -0.399}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64
(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP mult
isalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4: ['RF multisalida', 'MLP custom loss', 'Ridge', 'XGBoost single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.76), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.f
loat64(0.0), 'MLP custom loss': np.float64(0.24)}
[AVIS0] coll_shannon_z: todos los modelos tienen R2 ≤ 0 en meta-train, se usan pesos uniformes.

--- coll_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.519, 'RF single-target': -0.515, 'XGBoost single-target': -0.408, 'RF multisalida': -0.253,
'XGBoost multisalida': -0.372, 'RegressorChain': -0.76, 'MLP multisalida': -0.725, 'MLP custom loss': -0.57}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64
(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP mult
isalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4: ['RF multisalida', 'XGBoost multisalida', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.871), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.
float64(0.0), 'MLP custom loss': np.float64(0.129)}

--- bac_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.053, 'RF single-target': 0.03, 'XGBoost single-target': 0.037, 'RF multisalida': 0.063, 'XGB
oost multisalida': 0.005, 'RegressorChain': 0.039, 'MLP multisalida': 0.016, 'MLP custom loss': -0.052}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.158), 'XGBoost single-target': np.float64(0.1
96), 'RF multisalida': np.float64(0.332), 'XGBoost multisalida': np.float64(0.026), 'RegressorChain': np.float64(0.206), 'MLP multisal
ida': np.float64(0.082), 'MLP custom loss': np.float64(0.0)}
Top-4: ['RF multisalida', 'RegressorChain', 'XGBoost single-target', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.971), 'XGBoost multisalida': np.float64(0.029), 'RegressorChain': np.float64(0.0), 'MLP multisalida': n
p.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- fun_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.0, 'RF single-target': 0.011, 'XGBoost single-target': 0.024, 'RF multisalida': 0.012, 'XGB
oost multisalida': 0.038, 'RegressorChain': 0.021, 'MLP multisalida': -0.105, 'MLP custom loss': -0.032}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.101), 'XGBoost single-target': np.float64(0.2
27), 'RF multisalida': np.float64(0.115), 'XGBoost multisalida': np.float64(0.36), 'RegressorChain': np.float64(0.197), 'MLP multisal
ida': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4: ['XGBoost multisalida', 'XGBoost single-target', 'RegressorChain', 'RF multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.32
3), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.677), 'RegressorChain': np.float64(0.0), 'MLP multisalida':
np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- euk_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.015, 'RF single-target': 0.241, 'XGBoost single-target': 0.184, 'RF multisalida': 0.164, 'XGB
oost multisalida': 0.209, 'RegressorChain': 0.257, 'MLP multisalida': 0.239, 'MLP custom loss': 0.142}
Pesos media ponderada: {'Ridge': np.float64(0.01), 'RF single-target': np.float64(0.166), 'XGBoost single-target': np.float64(0.
127), 'RF multisalida': np.float64(0.113), 'XGBoost multisalida': np.float64(0.144), 'RegressorChain': np.float64(0.177), 'MLP multisa
lida': np.float64(0.165), 'MLP custom loss': np.float64(0.098)}
Top-4: ['RegressorChain', 'RF single-target', 'MLP multisalida', 'XGBoost multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.591), 'MLP multisalida': np.
float64(0.409), 'MLP custom loss': np.float64(0.0)}

```

```

--- oomy_shannon_z ---
R2 en meta-train por modelo: {'Ridge': 0.058, 'RF single-target': 0.064, 'XGBoost single-target': 0.087, 'RF multisalida': 0.096, 'XGB
oost multisalida': 0.046, 'RegressorChain': 0.067, 'MLP multisalida': 0.09, 'MLP custom loss': -0.02}
Pesos media ponderada: {'Ridge': np.float64(0.115), 'RF single-target': np.float64(0.127), 'XGBoost single-target': np.float64
(0.171), 'RF multisalida': np.float64(0.19), 'XGBoost multisalida': np.float64(0.09), 'RegressorChain': np.float64(0.131), 'MLP multisa
lida': np.float64(0.177), 'MLP custom loss': np.float64(0.0)}
Top-4: ['RF multisalida', 'MLP multisalida', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.06
8), 'RF multisalida': np.float64(0.493), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida':
np.float64(0.438), 'MLP custom loss': np.float64(0.0)}
[AVIS0] cerc_shannon_z: todos los modelos tienen  $R2 \leq 0$  en meta-train, se usan pesos uniformes.

--- cerc_shannon_z ---
R2 en meta-train por modelo: {'Ridge': -0.018, 'RF single-target': -0.092, 'XGBoost single-target': -0.112, 'RF multisalida': -0.072,
'XGBoost multisalida': -0.038, 'RegressorChain': -0.023, 'MLP multisalida': -0.17, 'MLP custom loss': -0.116}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64
(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP mult
isalida': np.float64(0.152), 'MLP custom loss': np.float64(0.125)}
Top-4: ['Ridge', 'RegressorChain', 'XGBoost multisalida', 'RF multisalida']
Pesos stacking convexo: {'Ridge': np.float64(0.59), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.
0), 'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.41), 'RegressorChain': np.float64(0.0), 'MLP multisalida':
np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- macro_order_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.072, 'RF single-target': 0.17, 'XGBoost single-target': 0.157, 'RF multisalida': 0.148, 'XGB
oost multisalida': 0.27, 'RegressorChain': 0.328, 'MLP multisalida': 0.247, 'MLP custom loss': 0.236}
Pesos media ponderada: {'Ridge': np.float64(0.044), 'RF single-target': np.float64(0.104), 'XGBoost single-target': np.float64
(0.096), 'RF multisalida': np.float64(0.091), 'XGBoost multisalida': np.float64(0.166), 'RegressorChain': np.float64(0.202), 'MLP mult
isalida': np.float64(0.152), 'MLP custom loss': np.float64(0.145)}
Top-4: ['RegressorChain', 'XGBoost multisalida', 'MLP multisalida', 'MLP custom loss']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.878), 'MLP multisalida': np.
float64(0.122), 'MLP custom loss': np.float64(0.0)}

--- earthworm_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.366, 'RF single-target': 0.558, 'XGBoost single-target': 0.52, 'RF multisalida': 0.453, 'XGB
oost multisalida': 0.605, 'RegressorChain': 0.55, 'MLP multisalida': 0.522, 'MLP custom loss': 0.536}
Pesos media ponderada: {'Ridge': np.float64(0.089), 'RF single-target': np.float64(0.136), 'XGBoost single-target': np.float64
(0.126), 'RF multisalida': np.float64(0.11), 'XGBoost multisalida': np.float64(0.147), 'RegressorChain': np.float64(0.134), 'MLP mult
isalida': np.float64(0.127), 'MLP custom loss': np.float64(0.13)}
Top-4: ['XGBoost multisalida', 'RF single-target', 'RegressorChain', 'MLP custom loss']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.91), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.f
loat64(0.0), 'MLP custom loss': np.float64(0.09)}

--- orib_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.128, 'RF single-target': 0.304, 'XGBoost single-target': 0.161, 'RF multisalida': 0.294, 'XGB
oost multisalida': 0.372, 'RegressorChain': 0.257, 'MLP multisalida': 0.383, 'MLP custom loss': 0.412}
Pesos media ponderada: {'Ridge': np.float64(0.056), 'RF single-target': np.float64(0.132), 'XGBoost single-target': np.float64
(0.07), 'RF multisalida': np.float64(0.127), 'XGBoost multisalida': np.float64(0.161), 'RegressorChain': np.float64(0.111), 'MLP mult
isalida': np.float64(0.166), 'MLP custom loss': np.float64(0.178)}
Top-4: ['MLP custom loss', 'MLP multisalida', 'XGBoost multisalida', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.0), 'XGBoost multisalida': np.float64(0.183), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.
float64(0.0), 'MLP custom loss': np.float64(0.817)}
[AVIS0] meso_species_richness_z: todos los modelos tienen  $R2 \leq 0$  en meta-train, se usan pesos uniformes.

--- meso_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.217, 'RF single-target': -0.208, 'XGBoost single-target': -0.236, 'RF multisalida': -0.121,
'XGBoost multisalida': -0.27, 'RegressorChain': -0.193, 'MLP multisalida': -0.179, 'MLP custom loss': -0.135}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64
(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP mult
isalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4: ['RF multisalida', 'MLP custom loss', 'MLP multisalida', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.537), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.
float64(0.0), 'MLP custom loss': np.float64(0.463)}
[AVIS0] coll_species_richness_z: todos los modelos tienen  $R2 \leq 0$  en meta-train, se usan pesos uniformes.

--- coll_species_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.606, 'RF single-target': -0.629, 'XGBoost single-target': -0.639, 'RF multisalida': -0.368,
'XGBoost multisalida': -0.724, 'RegressorChain': -0.699, 'MLP multisalida': -1.088, 'MLP custom loss': -0.824}
Pesos media ponderada: {'Ridge': np.float64(0.125), 'RF single-target': np.float64(0.125), 'XGBoost single-target': np.float64
(0.125), 'RF multisalida': np.float64(0.125), 'XGBoost multisalida': np.float64(0.125), 'RegressorChain': np.float64(0.125), 'MLP mult
isalida': np.float64(0.125), 'MLP custom loss': np.float64(0.125)}
Top-4: ['RF multisalida', 'Ridge', 'RF single-target', 'XGBoost single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.837), 'XGBoost multisalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multisalida': np.
float64(0.0), 'MLP custom loss': np.float64(0.163)}

--- bac_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.072, 'RF single-target': 0.032, 'XGBoost single-target': 0.02, 'RF multisalida': 0.038, 'XGB
oost multisalida': 0.037, 'RegressorChain': 0.021, 'MLP multisalida': -0.021, 'MLP custom loss': -0.107}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.217), 'XGBoost single-target': np.float64(0.1
37), 'RF multisalida': np.float64(0.255), 'XGBoost multisalida': np.float64(0.251), 'RegressorChain': np.float64(0.14), 'MLP multisalid
a': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4: ['RF multisalida', 'XGBoost multisalida', 'RF single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multisalida': np.float64(0.506), 'XGBoost multisalida': np.float64(0.494), 'RegressorChain': np.float64(0.0), 'MLP multisalida': n
p.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- fun_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.068, 'RF single-target': 0.047, 'XGBoost single-target': 0.021, 'RF multisalida': 0.05, 'XGB
oost multisalida': 0.007, 'RegressorChain': 0.053, 'MLP multisalida': 0.073, 'MLP custom loss': 0.068}
Pesos media ponderada: {'Ridge': np.float64(0.176), 'RF single-target': np.float64(0.122), 'XGBoost single-target': np.float64
(0.055), 'RF multisalida': np.float64(0.129), 'XGBoost multisalida': np.float64(0.017), 'RegressorChain': np.float64(0.136), 'MLP mult
isalida': np.float64(0.189), 'MLP custom loss': np.float64(0.175)}
Top-4: ['MLP multisalida', 'Ridge', 'MLP custom loss', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.224), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.

```

```

0), 'RF multitalida': np.float64(0.0), 'XGBoost multitalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multitalida': n
p.float64(0.534), 'MLP custom loss': np.float64(0.242)}

--- euk_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': -0.087, 'RF single-target': 0.228, 'XGBoost single-target': 0.185, 'RF multitalida': 0.174, 'XG
Boost multitalida': 0.299, 'RegressorChain': 0.296, 'MLP multitalida': 0.264, 'MLP custom loss': 0.2}
Pesos media ponderada: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.139), 'XGBoost single-target': np.float64(0.1
12), 'RF multitalida': np.float64(0.106), 'XGBoost multitalida': np.float64(0.182), 'RegressorChain': np.float64(0.18), 'MLP multitali
da': np.float64(0.16), 'MLP custom loss': np.float64(0.122)}
Top-4: ['XGBoost multitalida', 'RegressorChain', 'MLP multitalida', 'RF single-target']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multitalida': np.float64(0.0), 'XGBoost multitalida': np.float64(0.399), 'RegressorChain': np.float64(0.423), 'MLP multitalida': n
p.float64(0.178), 'MLP custom loss': np.float64(0.0)}

--- oomy_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.056, 'RF single-target': 0.112, 'XGBoost single-target': 0.113, 'RF multitalida': 0.098, 'XGB
oost multitalida': 0.087, 'RegressorChain': 0.129, 'MLP multitalida': -0.067, 'MLP custom loss': -0.087}
Pesos media ponderada: {'Ridge': np.float64(0.094), 'RF single-target': np.float64(0.189), 'XGBoost single-target': np.float64
(0.19), 'RF multitalida': np.float64(0.164), 'XGBoost multitalida': np.float64(0.146), 'RegressorChain': np.float64(0.218), 'MLP multitali
da': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4: ['RegressorChain', 'XGBoost single-target', 'RF single-target', 'RF multitalida']
Pesos stacking convexo: {'Ridge': np.float64(0.0), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.0),
'RF multitalida': np.float64(0.0), 'XGBoost multitalida': np.float64(0.137), 'RegressorChain': np.float64(0.863), 'MLP multitalida': n
p.float64(0.0), 'MLP custom loss': np.float64(0.0)}

--- cerc_asv_richness_z ---
R2 en meta-train por modelo: {'Ridge': 0.056, 'RF single-target': 0.021, 'XGBoost single-target': 0.044, 'RF multitalida': 0.053, 'XGB
oost multitalida': -0.029, 'RegressorChain': 0.021, 'MLP multitalida': -0.019, 'MLP custom loss': -0.073}
Pesos media ponderada: {'Ridge': np.float64(0.288), 'RF single-target': np.float64(0.106), 'XGBoost single-target': np.float64
(0.226), 'RF multitalida': np.float64(0.272), 'XGBoost multitalida': np.float64(0.0), 'RegressorChain': np.float64(0.109), 'MLP multitali
da': np.float64(0.0), 'MLP custom loss': np.float64(0.0)}
Top-4: ['Ridge', 'RF multitalida', 'XGBoost single-target', 'RegressorChain']
Pesos stacking convexo: {'Ridge': np.float64(0.488), 'RF single-target': np.float64(0.0), 'XGBoost single-target': np.float64(0.3
2), 'RF multitalida': np.float64(0.192), 'XGBoost multitalida': np.float64(0.0), 'RegressorChain': np.float64(0.0), 'MLP multitalida':
np.float64(0.0), 'MLP custom loss': np.float64(0.0)}

```

7.- Evaluación de todas las variantes sobre el holdout

Se calculan 5 formas de combinar los modelos disponibles, todas sobre las mismas filas de holdout:

1. **Media simple:** la de siempre, peso igual para todos.
2. **Media ponderada (R2):** pesos proporcionales al R2 de cada modelo en meta-train.
3. **Top-k:** media simple, pero solo entre los `K_TOP` mejores modelos.
4. **Mediana:** mas robusta a un modelo atípico (como Ridge) que la media.
5. **Stacking convexo:** combinacion lineal con pesos $w \geq 0$, $\sum(w) = 1$, optimizados en meta-train.

```

In [8]: holdout_predictions = {t: {} for t in TARGETS_TO_TEST}

for target in TARGETS_TO_TEST:
    if target not in variant_info:
        continue
    info = variant_info[target]
    model_labels = info["model_labels"]

    X_holdout = np.column_stack([predictions[target][lbl] for lbl in model_labels][idx_holdout]

    # Modelos individuales, recortados al holdout
    for i, lbl in enumerate(model_labels):
        holdout_predictions[target][lbl] = X_holdout[:, i]

    # 1. Media simple
    holdout_predictions[target]["Media simple"] = X_holdout.mean(axis=1)

    # 2. Media ponderada por R2
    holdout_predictions[target]["Media ponderada (R2)"] = X_holdout @ info["weighted_weights"]

    # 3. Top-k
    idx_top_k = [model_labels.index(lbl) for lbl in info["top_k_labels"]]
    holdout_predictions[target][f"Top-{len(idx_top_k)}"] = X_holdout[:, idx_top_k].mean(axis=1)

    # 4. Mediana
    holdout_predictions[target]["Mediana"] = np.median(X_holdout, axis=1)

    # 5. Stacking convexo (pesos ≥ 0, suman 1)
    holdout_predictions[target]["Stacking convexo"] = X_holdout @ info["convex_weights"]

```

8.- Comparativa: modelos individuales vs. las 5 variantes de combinacion

```

In [9]: COMBINATION_LABELS = {"Media simple", "Media ponderada (R2)", "Mediana", "Stacking convexo"}
COMBINATION_LABELS -= {lbl for lbl in holdout_predictions.get(TARGETS_TO_TEST[0], {}) if lbl.startswith("Top-")}

rows = []
for target in TARGETS_TO_TEST:
    if target not in variant_info:
        continue
    y_true_holdout = y_eval[target].values[idx_holdout]
    for label, preds in holdout_predictions[target].items():
        rmse = mean_squared_error(y_true_holdout, preds) ** 0.5
        rows.append({
            "target": target,
            "modelo": label,
            "R2": r2_score(y_true_holdout, preds),
            "RMSE": rmse,
            "MAE": mean_absolute_error(y_true_holdout, preds),
            "tipo": "combinacion" if label in COMBINATION_LABELS else "individual",

```

```

    })

if not rows:
    print("[AVISO] No hay predicciones de holdout disponibles todavía. "
          "Revisa que los modelos se hayan cargado (sección 3), que el smoke test haya "
          "pasado (sección 4) y que los pesos se hayan calculado (sección 6).")
    results_df = pd.DataFrame(columns=["target", "modelo", "R2", "RMSE", "MAE", "tipo"])
else:
    results_df = pd.DataFrame(rows).sort_values(["target", "R2"], ascending=[True, False]).reset_index(drop=True)

results_df

```

Out[9]:

	target	modelo	R2	RMSE	MAE	tipo
0	bac_asv_richness_z	XGBoost multisalida	-0.089010	0.855012	0.627395	individual
1	bac_asv_richness_z	XGBoost single-target	-0.090581	0.855628	0.626837	individual
2	bac_asv_richness_z	Ridge	-0.097045	0.858160	0.620105	individual
3	bac_asv_richness_z	Stacking convexo	-0.108851	0.862766	0.633342	combinacion
4	bac_asv_richness_z	Media ponderada (R2)	-0.116122	0.865590	0.634717	combinacion
...	...	...	...	...	...	...
268	orib_species_richness_z	Top-4	-0.138941	0.956782	0.712581	combinacion
269	orib_species_richness_z	XGBoost multisalida	-0.181075	0.974319	0.686219	individual
270	orib_species_richness_z	Stacking convexo	-0.218877	0.989788	0.722996	combinacion
271	orib_species_richness_z	MLP multisalida	-0.244074	0.999967	0.752825	individual
272	orib_species_richness_z	MLP custom loss	-0.271366	1.010875	0.731857	individual

273 rows × 6 columns

8.1.- Resumen agregado por target

Con muchos targets a la vez, `results_df` tiene demasiadas filas para leerla a ojo. Esta tabla se queda con una fila por target: el mejor modelo individual, la mejor variante de combinacion, cual de los dos gana, y si el mejor R2 de ese target es positivo (es decir, si hay alguna señal predictiva real) o no.

R2 <= 0 significa que ni el mejor modelo hace mejor que predecir siempre la media -- en esos targets el problema no es que metodo de ensamblado usar, sino que los modelos base no tienen señal útil que combinar.

```

In [10]: if results_df.empty:
    print("[AVISO] results_df esta vacio. Ejecuta primero las secciones 3-8.")
    summary_df = pd.DataFrame(columns=[
        "target", "mejor_individual", "R2_mejor_individual",
        "mejor_combinacion", "R2_mejor_combinacion", "gana_combinacion", "mejor_R2_global",
    ])
else:
    summary_rows = []
    for target in results_df["target"].unique():
        sub = results_df[results_df["target"] == target]

        individuales = sub[sub["tipo"] == "individual"].sort_values("R2", ascending=False)
        combinaciones = sub[sub["tipo"] != "individual"].sort_values("R2", ascending=False)

        mejor_individual = individuales.iloc[0] if not individuales.empty else None
        mejor_combinacion = combinaciones.iloc[0] if not combinaciones.empty else None

        r2_individual = mejor_individual["R2"] if mejor_individual is not None else float("-inf")
        r2_combinacion = mejor_combinacion["R2"] if mejor_combinacion is not None else float("-inf")

        summary_rows.append({
            "target": target,
            "mejor_individual": mejor_individual["modelo"] if mejor_individual is not None else None,
            "R2_mejor_individual": r2_individual,
            "mejor_combinacion": mejor_combinacion["modelo"] if mejor_combinacion is not None else None,
            "R2_mejor_combinacion": r2_combinacion,
            "gana_combinacion": bool(r2_combinacion > r2_individual),
            "mejor_R2_global": max(r2_individual, r2_combinacion),
        })

    summary_df = pd.DataFrame(summary_rows).sort_values("mejor_R2_global", ascending=False).reset_index(drop=True)

    n_targets = len(summary_df)
    n_gana_combo = int(summary_df["gana_combinacion"].sum())
    n_r2_positivo = int((summary_df["mejor_R2_global"] > 0).sum())

    print(f"Targets evaluados: {n_targets}")
    print(f"Targets donde alguna combinacion supera al mejor individual: "
          f"{n_gana_combo}/{n_targets} ({n_gana_combo / n_targets:.0%})")
    print(f"Targets con señal predictiva real (mejor R2 > 0): "
          f"{n_r2_positivo}/{n_targets} ({n_r2_positivo / n_targets:.0%})")
    print(f"Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): "
          f"{n_targets - n_r2_positivo}")

summary_df

```

Targets evaluados: 21
Targets donde alguna combinacion supera al mejor individual: 4/21 (19%)
Targets con señal predictiva real (mejor R2 > 0): 15/21 (71%)
Targets sin señal predictiva (mejor R2 ≤ 0, ningún modelo le gana a la media): 6

		target	mejor_individual	R2_mejor_individual	mejor_combinacion	R2_mejor_combinacion	gana_combinacion	mejor_R2_global
0	macro_order_richness_z		XGBoost multisalida	0.509020	Top-4	0.450304	False	0.509020
1	earthworm_richness_z		XGBoost multisalida	0.508708	Stacking convexo	0.496985	False	0.508708
2	cerc_asv_richness_z		XGBoost multisalida	0.404936	Mediana	0.324205	False	0.404936
3	macro_shannon_z		RegressorChain	0.372242	Stacking convexo	0.370340	False	0.372242
4	earthworm_shannon_z		RegressorChain	0.366510	Top-4	0.351522	False	0.366510
5	oomy_asv_richness_z		RF multisalida	0.302448	Stacking convexo	0.279431	False	0.302448
6	nematode_shannon_z		XGBoost multisalida	0.280802	Stacking convexo	0.238977	False	0.280802
7	euk_asv_richness_z		RF single-target	0.219539	Mediana	0.217647	False	0.219539
8	cerc_shannon_z		RF multisalida	0.121626	Mediana	0.069647	False	0.121626
9	oomy_shannon_z		MLP multisalida	0.109946	Stacking convexo	0.084285	False	0.109946
10	euk_shannon_z		RF multisalida	0.103819	Mediana	0.057647	False	0.103819
11	meso_species_richness_z		RF single-target	0.053125	Mediana	0.043725	False	0.053125
12	coll_shannon_z		RF multisalida	-0.058985	Mediana	0.032299	True	0.032299
13	meso_shannon_z		RegressorChain	0.023491	Mediana	-0.058045	False	0.023491
14	orib_shannon_z		MLP custom loss	-0.001389	Media ponderada (R2)	0.011586	True	0.011586
15	coll_species_richness_z		RF multisalida	-0.069098	Stacking convexo	-0.015200	True	-0.015200
16	fun_shannon_z		Ridge	-0.045406	Mediana	-0.095235	False	-0.045406
17	orib_species_richness_z		RF multisalida	-0.054500	Mediana	-0.070825	False	-0.054500
18	fun_asv_richness_z		Ridge	-0.087334	Stacking convexo	-0.086309	True	-0.086309
19	bac_asv_richness_z		XGBoost multisalida	-0.089010	Stacking convexo	-0.108851	False	-0.089010
20	bac_shannon_z		RegressorChain	-0.129290	Top-4	-0.244984	False	-0.129290

9.- Visualización

Barras de R2 por modelo y target sobre el holdout. Las variantes de combinacion se resaltan con borde negro para distinguirlas de los modelos individuales.

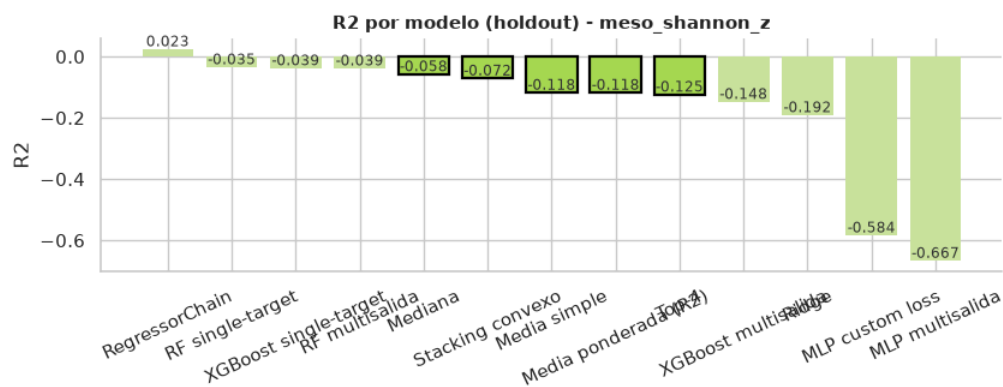
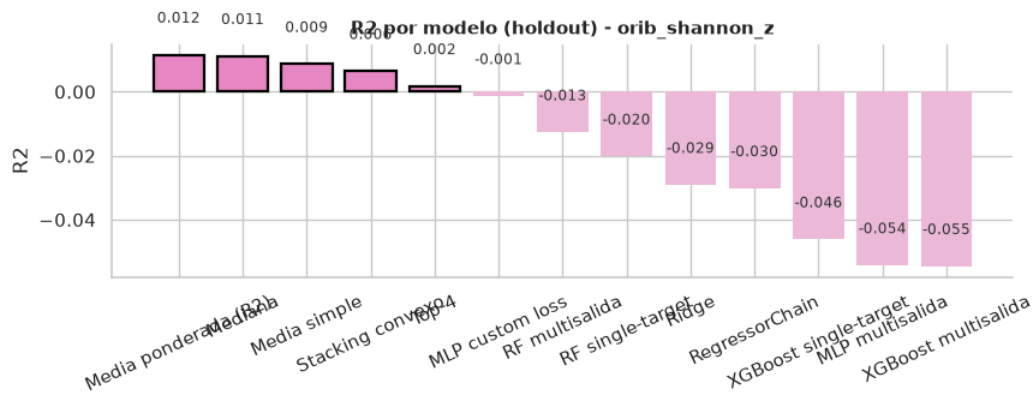
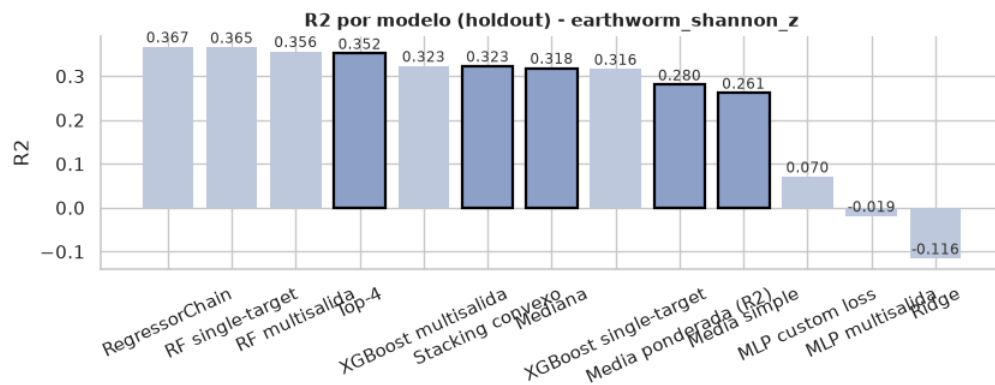
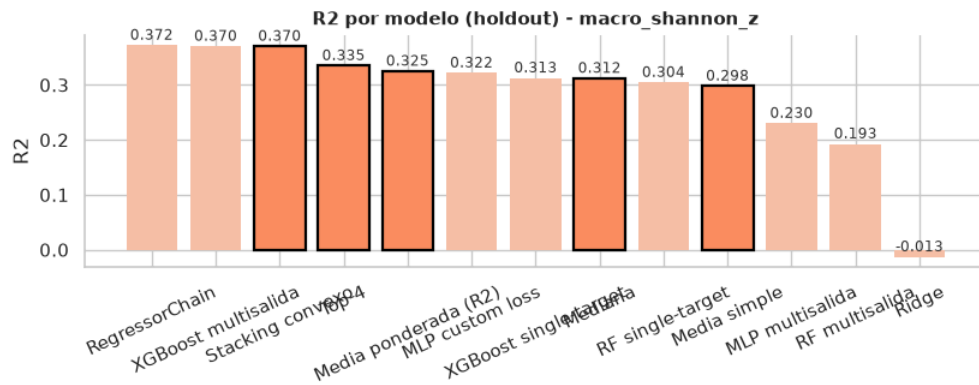
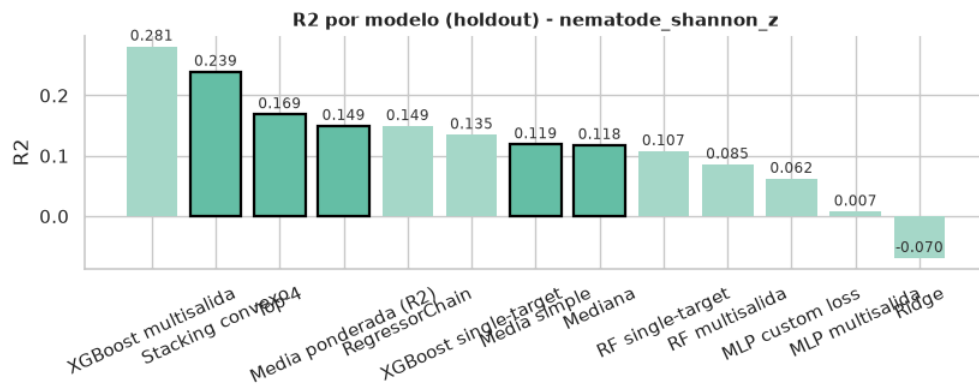
```
In [11]: if results_df.empty:
print("[AVISO] results_df esta vacio, no hay nada que graficar. Ejecuta primero las secciones 3-8.")
else:
    targets_present = [t for t in TARGETS_TO_TEST if t in results_df["target"].unique()]
    palette = sns.color_palette("Set2", n_colors=max(len(targets_present), 3))
    TARGET_COLORS = dict(zip(targets_present, palette))

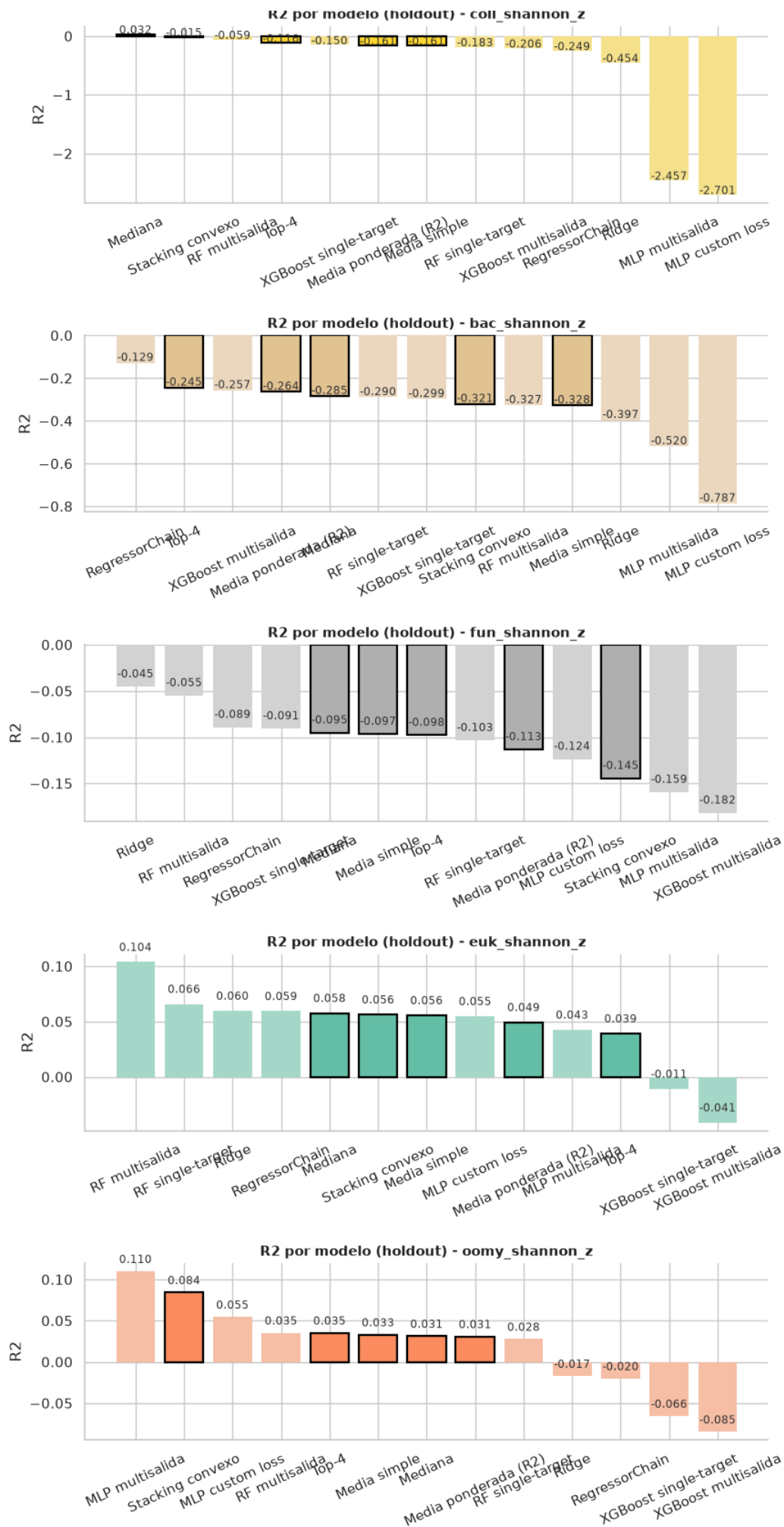
    fig, axes = plt.subplots(len(targets_present), 1, figsize=(9, 3.4 * len(targets_present)), squeeze=False)

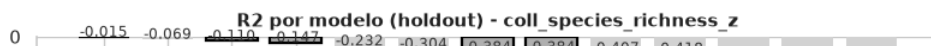
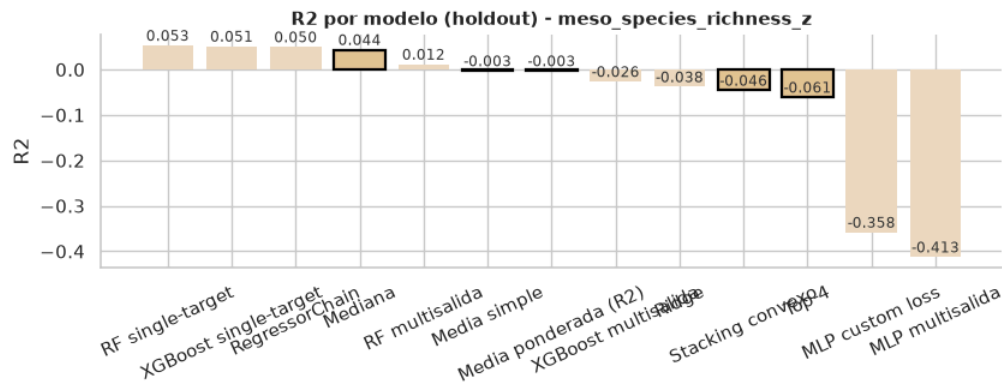
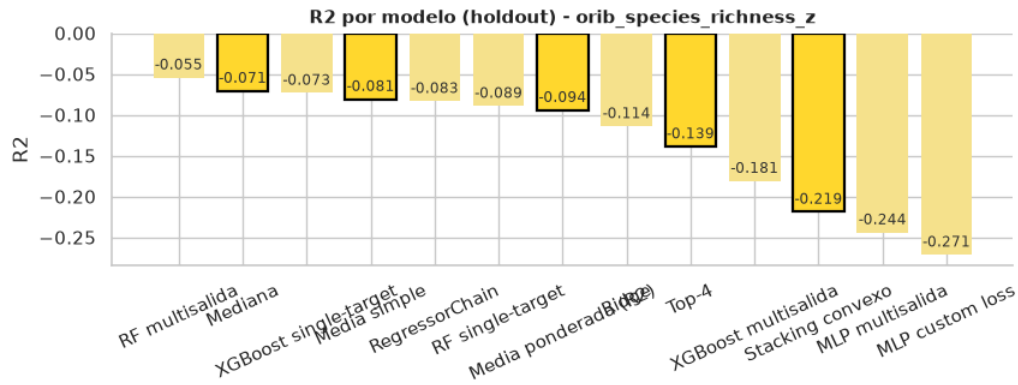
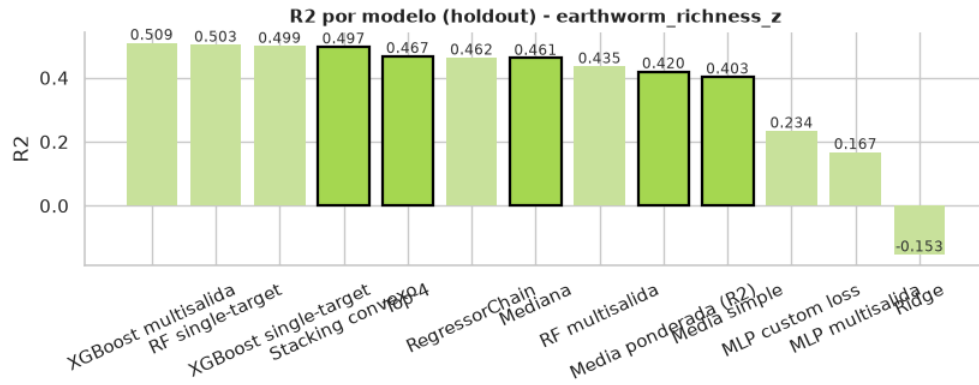
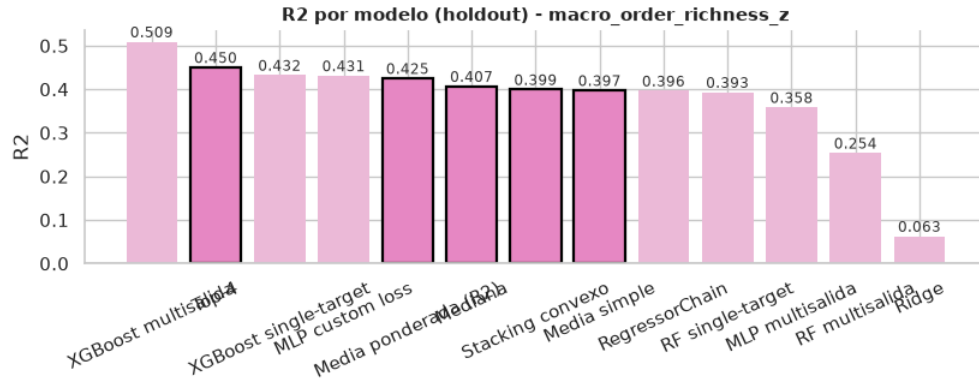
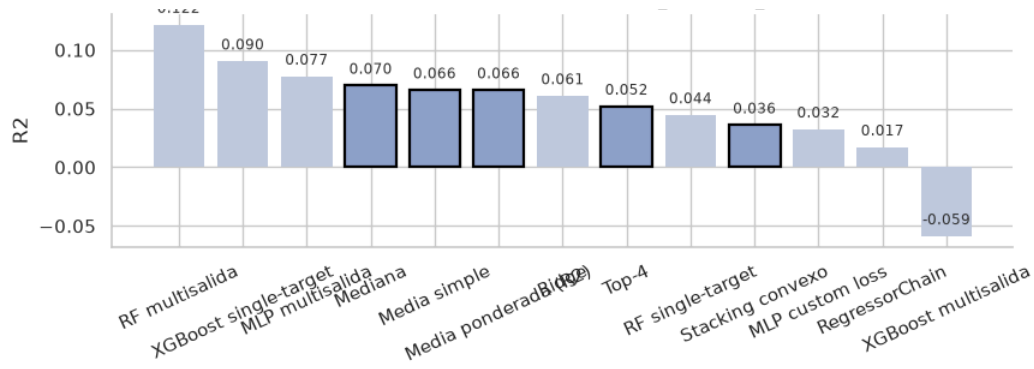
    for ax, target in zip(axes[:, 0], targets_present):
        sub = results_df[results_df["target"] == target].sort_values("R2", ascending=False)
        base_color = TARGET_COLORS[target]
        is_highlighted = sub["tipo"] != "individual"
        colors = [base_color if h else sns.light_palette(base_color, n_colors=3)[1] for h in is_highlighted]
        edgecolors = ["black" if h else "none" for h in is_highlighted]
        linewidths = [1.6 if h else 0 for h in is_highlighted]

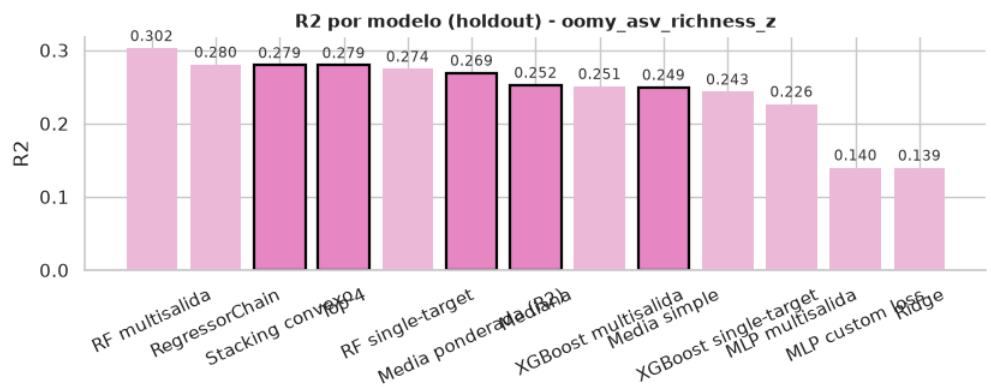
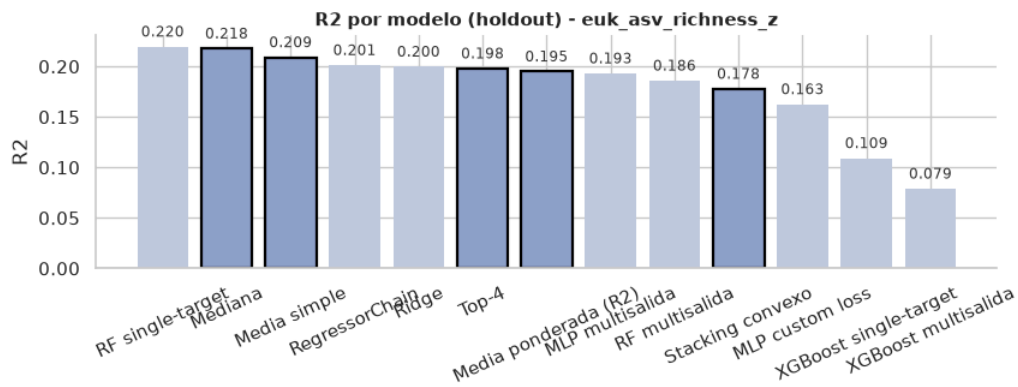
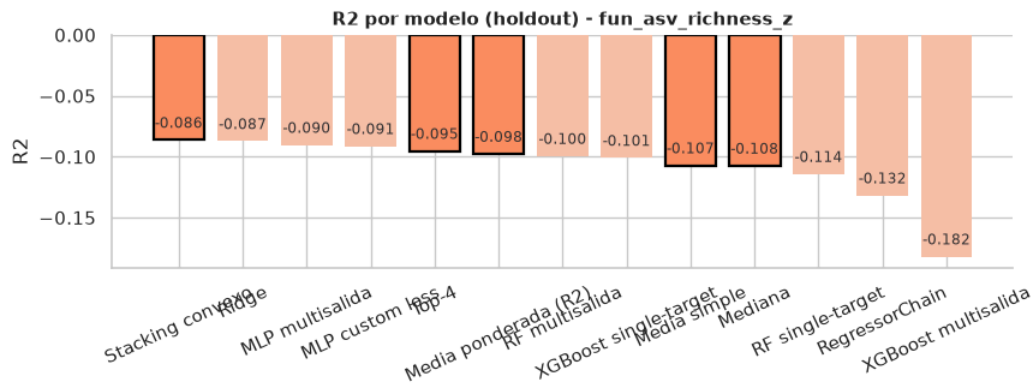
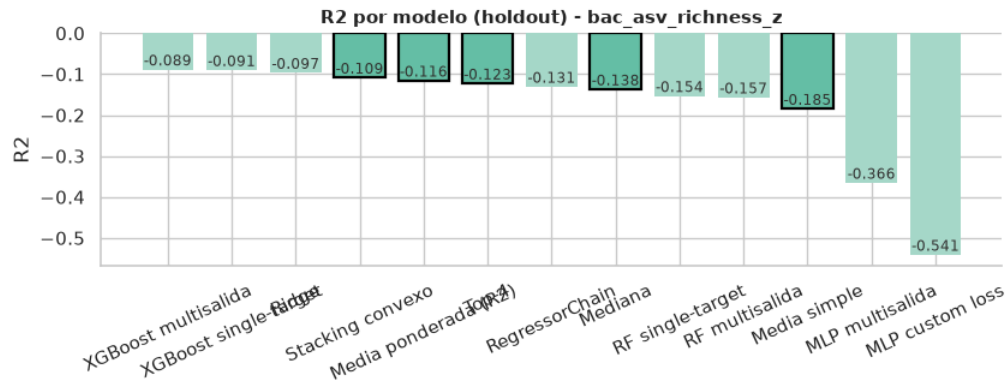
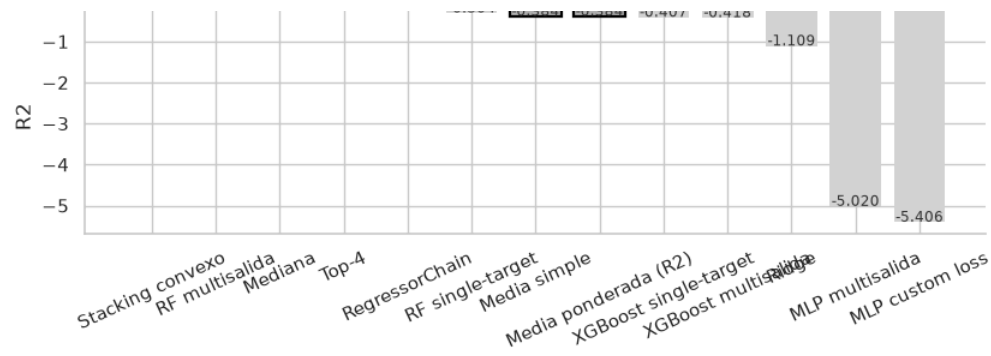
        bars = ax.bar(sub["modelo"], sub["R2"], color=colors, edgecolor=edgecolors, linewidth=linewidths)
        for bar, r2 in zip(bars, sub["R2"]):
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f"{r2:.3f}",
                    ha="center", fontsize=8.5, color="#333333")
        ax.set_title(f"R2 por modelo (holdout) - {target}", fontsize=11, fontweight="bold")
        ax.set_ylabel("R2")
        ax.tick_params(axis="x", rotation=25)
        ax.spines["top"].set_visible(False)
        ax.spines["right"].set_visible(False)

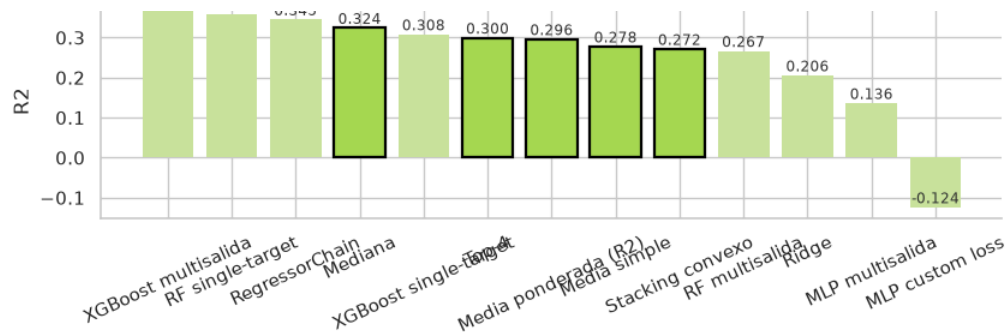
    fig.tight_layout()
    fig.savefig("output/r2_comparativa_sin_modelos.png", dpi=300, bbox_inches="tight")
    plt.show()
```











10. Conclusiones

Sobre el mismo holdout de 20 filas, las cinco variantes de combinación probadas (media simple, media ponderada por R^2 , top-k, mediana y stacking convexo) no logran superar de forma consistente al mejor modelo individual. Solo "ganan" en 4 de los 21 targets (19%) (`coll_shannon_z` , `orib_shannon_z` , `coll_species_richness_z` y `fun_asv_richness_z`) y en los cuatro casos se trata de targets sin señal predictiva real (el mejor R^2 individual ya era ≤ 0), así que más que una mejora es un empate entre modelos igualmente poco útiles. En los 15 targets donde sí hay señal ($R^2 > 0$), ninguna combinación bate al mejor individual, y los dos targets prioritarios lo confirman: en `earthworm_shannon_z` , RegressorChain ($R^2=0.3665$) supera a la mejor combinación, Top-4 ($R^2=0.3515$) y en `earthworm_richness_z` , XGBoost multisalida ($R^2=0.5087$) supera al Stacking convexo ($R^2=0.4970$).

El comportamiento del **stacking convexo** explica en parte este resultado: en la mayoría de targets concentra entre el 68% y el 100% del peso en un solo modelo (casi siempre XGBoost multisalida), dejando el resto en cero. En la práctica no combina modelos, sino que selecciona uno solo de ahí que como mucho iguale al mejor individual, pero nunca lo mejore.

La **media ponderada**, en cambio, es mucho más conservadora: ningún modelo, ni siquiera el más débil (Ridge o MLP), baja de pesos de 0.03–0.12, por lo que apenas penaliza a los candidatos flojos. En los targets sin señal (`meso_shannon_z` , `coll_shannon_z`), todos los R^2 en meta-train son negativos y la media ponderada cae, por diseño, a pesos uniformes.

Tampoco hay un patrón claro de qué variante conviene: entre los cuatro targets donde alguna combinación "gana" aparecen tres variantes distintas (Mediana, Media ponderada y Stacking convexo), sin relación aparente con el tipo de target.

Y quedan dos puntos sin verificar antes de sacar conclusiones firmes: por un lado, esta ejecución tuvo `SHOW_ENSEMBLE_MEMBERS = False` , así que no hay datos sobre si los miembros internos de cada ensamblado multisalida son consistentes entre sí o si alguno desentona; por otro, solo se ha probado con un único `RANDOM_STATE` , y con un holdout de apenas 20 filas, diferencias de una o dos centésimas de R^2 , como las de los cuatro "empates" señalados, están dentro del ruido esperable al cambiar de semilla.

En conjunto: con los datos disponibles, ninguna de las cinco variantes de combinación aporta una mejora real sobre el mejor modelo individual por target. Para los dos targets prioritarios, la recomendación es mantener el modelo individual ganador (RegressorChain para `earthworm_shannon_z` , XGBoost multisalida para `earthworm_richness_z`), y reservar el stacking convexo o la media ponderada solo si en el futuro, con más semillas o más datos, empiezan a mostrar una ventaja sostenida.